

Troll Farm: how the four top players play, and how we found out

A report for the project owner · 28 August 2026 · written from the reconstruction work of the night of 27–28 August

1. What this report is

You asked for the algorithms of the four best players of the Troll Farm competition, described well enough that a program could be written from the description, using every means available: what the players published, our own records of their games, and our earlier work. This report collects all the findings in plain words. Every number comes with the size of the sample it was measured on, and every guess is called a guess.

The four players, in the order of the ranking on 27 August at 19:50 UTC, are **delineate** (rating 30.89), **norxondor_gorgonax** (29.66), **Bubaptik** (27.90) and **MSz** (27.72). Our own bot stood at about 19–21.

The work behind this report was done in about an hour and a half by eight parallel workers (one searched the internet, one read our own earlier studies, one measured the players' habits in our records, one replayed their games turn by turn and tested candidate rules, four wrote one document per player) and three reviewers (one re-read the four documents against their sources, two read this report). The full documents, one per player, are in the repository under `local_claude_1/reconstructions/`; this report is the readable summary of all of them.

2. The game in plain words

Troll Farm is a two-player game played by programs. Each player has a **shack** (a home base) on a small map of grass, water, rock and iron cells. The map is symmetric, so both players start in equal conditions. A game lasts up to 300 turns; on every turn each program gives one command to each of its **trolls** (its workers).

Trees grow on the map in four kinds — plum, lemon, apple and banana. A tree grows in size (up to 4) and bears up to three fruits at a time. A troll standing on a tree can **harvest** a fruit; a troll can also **chop** a tree, and when the tree falls it yields **wood** equal to its size (up to 4 pieces). Trolls carry what they collect back to the shack and **drop** it there. A fruit dropped at the shack is worth **1 point**; a piece of wood is worth **4 points**. Trolls can also **plant** a fruit as a seed on an empty grass cell (a new tree grows there), **pick** a fruit from the shack to use as a seed, and **mine** iron next to an iron cell.

Every player starts with one troll and a random small stock of each fruit and of iron (2 to 10 of each). A player can **train** more trolls at the shack. A new troll has four **talents**, each a number: *speed* (cells moved per turn), *carry* (items carried at once), *harvest* (fruits taken per harvest; 0 means it cannot harvest at all) and *chop* (damage per chop; 0 means it cannot chop). We write the talents in that order, for example 2/4/0/3. The price of a new troll is paid from the shack's stock and grows with the square of each talent: plums for speed, lemons for carry, apples for harvest, iron for chop, plus one extra of each for every troll the player already has. So a troll with carry 4 costs sixteen lemons plus the count of existing trolls — expensive, which is why the best players grow a **lemon orchard** next to their shack to pay for it.

A tree's toughness depends on its kind and size: plums and lemons are medium, apples are tough, bananas are soft (a freshly planted banana falls to a single blow from a strong chopper; a full-grown apple takes many). Bananas also regrow fastest. Wood is where most of the points are: the top players make 80–93 % of their score from wood.

The game's rules engine is called the **referee**; it applies the commands, grows the trees, and keeps

the score. On the public ladder every program plays a batch of games against others and receives a **rating** (about 30 for the best, about 20 for ours) and a rank.

3. Who the four are

Rank	Player	What kind of program it is	How much of it we know
1	delineate	A neural network trained by practice games (explained in section 6). It does not follow written rules.	Its author described its design in detail. Its habits are measured. There are no rules to copy, only behaviour to imitate.
2	norxondor_gorgonax	A fast rule-based program with two clear phases: first produce (grow trolls and an orchard), then deforest (chop everything).	The author never published anything, but its training plan and its phase switch are recovered almost exactly from its games. How it picks which tree to chop is not recovered.
3	Bubaptik	A rule-based program that grows three to four trolls, unusually fast ones, and switches to chopping the moment its last troll is trained.	It was not in the contest's top league (probably a later entrant) and left no trace on the internet. Everything comes from 191 of its games in our records.
4	MSz	A "farm-first" program: it trains a cheap helper on turn 1, plants an orchard around the shack, then buys one or two heavy lumberjacks and harvests fruit all game.	Its author published nothing about this game. Its plan and timing are measured; whether it runs a search inside (trying out possible futures before choosing) is a guess.

Our own bot belongs to a different family: it keeps **two trolls** for the whole game (the second one cannot harvest), chops wild trees, and plants little. That design was the contest's number three (a player called yamo, whose published write-up we reproduced in July). The contest's top ten splits cleanly into these two families: two-troll choppers, and three-to-four-troll "build-up" economies with an orchard next to the shack.

4. Where the findings come from

Four independent sources were used, and each finding below says which of them supports it.

1. **What the players published.** The winner, delineate, wrote a detailed description of his program (a public note on GitHub, called a gist, and two forum posts). The contest's author, euler-scheZahl, published a statistics page with measured habits of all 69 top-league players (trolls trained, trees planted, score over time). Eleven other top-league players described their programs in the contest's feedback thread; the ones of the same family as norxondor, Bubaptik and MSz explain mechanisms those three never wrote down. Nothing was found from norxondor_gorgonax (no account anywhere), from Bubaptik (not in the contest), or from MSz about this game. All sources are listed with links in section 12 and archived verbatim in the repository.
2. **Our own records.** We have collected about 24,000 games from the public ladder, among them 191 to 223 games of each of the four players. Each recorded game contains every command each program gave, and the referee's own log of what happened ("troll 3 moved to (10,5)", "planted a plum", "harvested 2 lemons"). From these we measured each player's habits: when it trains, what it buys, what and where it plants, what it chops, in which turns, with what result. Positions and outcomes are read from the referee's own log, so they are exact in all 848 of these games.

3. **Replaying the games and testing rules.** We rebuilt the exact state of every turn of 784 games (the map, every tree's size and health, every troll's position and load, both stocks) by running the commands through our own copy of the rules and checking it against the recording (no disagreement in any of them, except where the referee itself picks one of two equal paths at random). With the exact states we could list, at every decision, what the troll *could* have done and test which simple rule predicts what it *did*: "go to the nearest fruit tree", "plant on the cell closest to home", "train as soon as you can pay", and so on. A rule that predicts 90 % of the decisions is close to the truth; one that predicts 30 % is not the rule the program uses.
4. **Our earlier work.** In July the project had already recovered norxondor's training plan and had tried to imitate the top players in several ways. Those attempts left a warning that this report repeats in section 10.

5. What the four have in common

These facts hold for all four players, measured on 191–223 games each (182–215 of them replayed in full), and they are the heart of the report.

- **They grow to three or four trolls, and the later ones are lumberjacks.** The third and fourth trolls have carry 3 or 4, chop 2 or 3, and little or no harvest power. delineate ends its games with 2.9 trolls on average and wins 91 % of the games in which it reaches four; norxondor ends with 3.5; MSz and Bubaptik reach four in 38–45 % of games.
- **They train the moment the price can be paid.** In 88–100 % of all trainings the troll was bought within one turn of its price becoming affordable — on that very turn in about 60 % of cases for delineate and Bubaptik and in 99 % for norxondor and MSz. There is no waiting for a better moment — the only question is which troll to save for.
- **They pay for those trolls with an orchard planted next to the shack.** All four plant 29–40 trees per game (our bot plants 10, almost all in the last fifty turns). The cell they plant on obeys one rule in 78–90 % of all plantings: *the free cell that minimises the distance to the shack plus the distance to the troll*. Lemons and plums come first for three of the four (they pay for carry and speed; MSz plants bananas from the start as well), bananas later (the fastest-growing and softest tree, so the quickest wood).
- **Their wood comes from their own trees, late in the game.** For norxondor and MSz the first wood reaches the shack only around turn 100–120; delineate and Bubaptik bring home a little wood from their early raids (first wood around turn 25), but for all four the chop rate climbs steeply only after turn 100–150. norxondor and Bubaptik switch explicitly from producing to chopping; MSz keeps harvesting fruit while its lumberjacks chop; delineate simply chops more as it gets more trolls.
- **No simple formula explains which tree they chop.** Our bot's rule (wood per turn of the round trip) predicts only 30–42 % of their choices — and just 16–30 % when the rule rates several trees equally and must pick one at random — and no other simple rule does much better. What the games show instead are phases and roles: the starting troll farms at the shack, the lumberjacks only chop, and for three of the four the early game has a raid on the opponent's fresh trees (MSz does not raid). This is the largest open gap for writing a program, and it is named honestly in each player's document.

6. delineate — the winner

What it is. delineate's program is a **neural network**: a large table of numbers that turns a picture of the board into a choice of action, adjusted over a great many practice games (the author gives no count; playing against itself is implied) until good choices became habits. Its author wrote the design down. The network has about 101,000 numbers, looks at the board as 104 layers of an 11×22 grid (where the trees are, their sizes, where the trolls are, what each carries, the stocks, the turn, and so on), and each turn does the following: one pass to choose a *training target* (which troll to save for next, out of 144 possible talent combinations, or "done training"), then one pass per

troll to score every action that troll could take, then a small search to make sure two trolls do not walk into the same cell. It does not look ahead in the game at all. It answers in 2–3 milliseconds. (Author's gist and forum posts; high confidence.)

How it was made to work. The author's real "rules" are the way he taught the network, in five stages: first, build one specific troll (rewarding every step that brings the required stock closer); then random targets; then a random number of trolls to build and the true score difference, with a small immediate reward for every piece of wood dropped at the shack; then a separate part of the network that chooses the training target, trained on the score alone (when the network was allowed to choose its own targets under the step rewards it cheated: it built many cheap trolls to collect the rewards); finally, everything trained together once more on the score alone. A first plain attempt without these stages "could never learn to mine iron" and stayed around 25th place. (Author's gist.)

What it does, measured. On 223 of its ladder games: it wins 78 %, scoring 415 to its opponents' 253. Wood is 93 % of its points (98 pieces per game: 75 from its own trees, 12 from the opponent's, 11 wild). It buys its second troll at about turn 6, a balanced worker (2/2/2/2 or 2/2/1/2), by a simple rule: each talent is set to the highest level the shack can pay right now (exact in 22 of 26 games checked). A third troll (usually carry 4, chop 3) comes in 56 % of games, typically at turn 111 (the median, that is the middle value over those games); a fourth (usually 3/4/1/3) in 27 % at 144. The starting troll stays near the shack as a farmer (harvest, drop, plant); every trained troll spends half to three quarters of its action turns (moves not counted) chopping. It plants 40 trees per game — lemons first, bananas after turn 100 — on the cell closest to the shack and the troll (90 % of plantings). In the first fifty turns 84 % of its chop targets are on the opponent's half of the map, 56 % within two cells of the opponent's shack: it cuts down the enemy's young trees. In the last fifty turns it cuts its own banana farm, trees grown to full size. It has no end-game switch: it plants until turn 296 and drops wood until the last turn. (Our records; the game author's statistics agree.)

What a program would need. There are two honest routes. One is to repeat the author's recipe: build the same kind of network and the five training stages — a project of days or weeks, not a rule to write. The other is to imitate the measured behaviour with rules: a fixed sequence of trolls to buy that follows the measured talents, the "closest to shack and troll" planting rule, an early raid on the opponent's fresh trees, and chopping one's own full-grown trees late. The document `define/ALGORITHM.md` writes this imitation out step by step and marks what it cannot reproduce (its guesses are the exact triggers of the raid and of the switch to its own trees).

7. norxondor_gorgonax — produce, then deforest

What it is. A rule-based program, and a fast one: it prints its own timing every turn, 0.13 milliseconds on average over 63,945 recorded turns, so it does not run a search of possible futures — it applies rules. Its author published nothing; everything below is measured on 218 of its ladder games, where it wins 67 %, scoring 345 to 272. (Our records; the game author's contest statistics show the same plan; against the contest's stronger field it won 56 %.)

The plan, in phases. The program lives in one of two modes, which it even announces in its chat message: **P** for "produce" and **D** for "deforest" (our reading of the two letters). In P mode it plants and harvests, builds its trolls, and hardly chops (7 chops in the first 100 turns; the first wood reaches the shack at median turn 97). It switches to D at median turn 153 (the middle half of games between 124 and 173), one turn after its last training in 62 % of games. In D mode it never trains again and every troll chops; it reaches the highest chop rate of the four (12–13 chops per 10 turns), stops harvesting around turn 222, and in 48 of 218 games it leaves the map with no tree at all.

The training plan — recovered exactly. It buys a troll as soon as the shack can pay a minimum for the next troll in line: the minimums are 2/2/1/1 for the second troll, 2/3/1/2 for the third, 2/3/0/3 for the fourth and 2/4/0/3 for the fifth. When it buys, each talent is raised to the most the shack can pay at that moment, up to caps (speed 4, carry 5, harvest 2, chop 2 for the first two purchases and 3 after). This rule reproduces 441 of 443 recorded purchases, and the purchase happens on the first affordable turn in 439 of 444. It buys the second troll on turn 1 whenever the starting stock already covers the minimum (76 of 184 games); over all games the second troll

comes at median turn 9–14, the third around turn 100–106, the fourth around 132–138. It mines iron only when iron is what is missing for the next troll (all 1,036 mining trips), starting around turn 72.

Planting. It plants very close to home: median one cell from the shack, never more than four. 35 % lemons, 32 % plums, 26 % bananas. The cell follows the “closest to shack and troll” rule in 87 % of plantings. Its signature move is **plant-and-cut**: it plants a banana on the cell next to the shack and chops it the next turn, at size 1 — one fruit point becomes four wood points, without walking anywhere. In 184 games, 1,116 of its own bananas were cut at size 1, and 2,407 of all its own-tree cuts came within four turns of planting.

Chopping and harvesting. Which tree it chops is not recovered: the best rule we found (wood per turn of the trip) predicts 41 % of its choices. What is measured: it prefers plums (37 %) and lemons (33 %) over bananas (14 %); 30 % of its chops fall on trees the opponent planted and 36 % are nearer the opponent’s shack than its own — it is the most opponent-directed of the four; early in the game 59 % of its chops are on the opponent’s trees. Harvesting follows a throughput rule (fruits gained per turn of the trip) in 59 % of choices, preferring the kinds it still needs for the next troll.

What a program would need. The training rule, the phase switch, the planting rule and the plant-and-cut move can be written down as they stand; the document `norxondor_gorgonax/ALGORITHM.md` does so in pseudo-code (a step-by-step recipe written like a program, in words). What must still be guessed: how the program orders its chop targets, the meaning of the second letter of its chat message, and what exactly triggers the switch in the 38 % of games where it is not “one turn after the last training”.

8. Bubaptik — the fast lumberjacks

What it is. A rule-based build-up program that was not in the contest’s top league; no write-up exists anywhere. Our records hold 34 versions of it; the newest (191 games) wins 65 %, scoring 295 to 310 — it wins narrowly and loses big, 74 % against two-troll opponents but 32 % against opponents rated 28 or more. Wood is 91 % of its points, and it brings home the least wood of the four (69 pieces per game; MSz and norxondor 80, delineate 98), although MSz gives fewer chop commands. (Our records only.)

The plan. It buys its second troll on turn 2 in 154 of the 186 first purchases we could trace (83 %), and the rule is exact in 147 of those 154 cases: each talent is set to the highest level the starting stock can pay. Its third, fourth and fifth trolls are unusual: **speed 4**, carry 3, chop 2 or 3 (4/3/h/c, where the harvest level follows the apple stock and chop is 3 only if there is enough iron — the third troll has chop 2 in 114 of 154 cases). Speed 4 is paid in plums (sixteen plus the number of trolls), so it is a **plum** farmer: 47 % of what it plants is plum. The third troll comes at about turn 115, the fourth at 150, the fifth at 164, nearly always on the first affordable turn (139 of 147 third trolls). When plums are scarce it falls back to a slow troll (1/3/h/c) in about one purchase in five.

The switch. Its wood phase begins with a hard, global switch at its *last* training: chops per turn jump from 0.03 to 0.69, mining stops, and it starts picking bananas from the shack to plant as its quick wood crop (890 such picks after the switch, 18 before). Games in which it never gets past two trolls never switch.

Roles and rules. The starting troll farms (harvest, drop, plant); the second troll, a small hybrid (speed 2, carry 2; it can both harvest and chop), does the early denial — chopping the opponent’s young trees so they never pay off (half of its early chops are on the opponent’s trees); the speed-4 trolls chop 69–77 % of their action turns. Planting follows the “closest to shack and troll” rule in 84 % of cases, a little farther out than the others (2.4 cells on average). Harvesting goes to the nearest fruit tree without one of its own trolls on it (54 %). Early chopping goes to the nearest tree on the opponent’s half (56 %); the choice of tree in the middle and late game is not recovered. It is the only one of the four that moves “to a destination” rather than one step at a time, so its intentions are readable in the replay.

What a program would need. The training rule, the switch and the roles are measured; the document Bubaptik/ALGORITHM.md writes them out. Guesses: why it stops buying trolls where it does (14 affordable fifth trolls were not bought), what makes it wait for five lemons on turn 2, and its harvest and plant kind choices.

9. MSz — the fruit farmer with heavy lumberjacks

What it is. A build-up program of the “farm first” kind, by a player who won several earlier contests with programs built on an exact simulation and a search over sets of actions; whether this program searches is a guess, because he published nothing about it. Our records hold 216 of its games: it wins 57 %, scoring 399 to 333. It is the only one of the four that harvests fruit all game long — 129 fruits per game, 78 fruit points (the others 27–34), and wood is “only” 80 % of its score. It is also the only apple farmer: 11 % of its plants are apples, half of them next to water (apples regrow much faster there), and apples are a third of all the fruit it harvests. (Our records; the game author’s contest statistics agree closely.)

The plan. It trains on **turn 1** in 196 of the 203 games we replayed in full (97 % of all its games), by an exact rule that held in all 196: speed 2 if there are at least 5 plums in the starting stock, carry 2 if at least 5 lemons, harvest 2 if at least 5 apples and at least 5 lemons (that is, only together with carry 2), chop always 1 — the cheapest useful helper the stock allows. In turns 2–25 both trolls plant a two-ring orchard around the shack (91 % of its plantings are one or two cells from the shack, none beyond six). Then both harvest lemons first (60 % of harvest trips in the first fifty turns) and mine one iron per trip (first mining at turn 20), holding plums and apples flat at exactly the level the next troll needs, while lemons pile up. The third troll — 2/4/1/2 or 2/4/1/3, always carry 4 — is bought on the first affordable turn (441 of 444), at median turn 97; the fourth, 2/4/0/3, in 38 % of games at turn 128 — for that one the program waits for the twelfth iron that chop 3 costs rather than settle for chop 2 (in 18 of the 21 cases where iron was the last thing missing). It never buys a fifth. The plan is fixed and it waits for it: in 200 of 444 purchases a bigger troll was affordable and not taken. In 16 % of games the third troll never comes (the lemons never reach 18, or plums and apples never meet their bill) and it wins only 12 % of those.

Wood and fruit. Chopping starts late (first wood at turn 116) and stays at home: 118 chop commands per game, the fewest of the four (though Bubaptik brings home less wood), 78 % nearer its own shack, on trees grown to full size (71 % size 4, 26–37 turns old). It does not raid: 4 % of its early chops are near the enemy’s shack, and in the contest its opponents scored as much as it did. Its farm keeps running to the end (harvesting until turn 298, apples as the late crop).

What a program would need. The turn-1 rule, the orchard, the funding order, the fixed sequence of trolls and the roles are measured and written out in MSz/ALGORITHM.md. The search it runs inside (if any), how it orders chop targets (no rule predicts more than about a third of them), and the composition of the orchard as an intention rather than an outcome are guesses.

10. What we could not recover, and a warning

Not recovered, for all four: the rule that orders the trees to chop, and the rule that chooses which kind of fruit to plant. For delineate there is no rule at all; for the other three there probably is one, but none of the simple candidates fits. **Not recovered, per player:** norxondor’s tie-breaks (what it does when two trees score equally) and the meaning of part of its chat message; MSz’s search and evaluation; the limit on how many trolls Bubaptik buys, and what makes it fall back to a slow troll.

The warning. In July this project built a program from rules fitted to norxondor’s games (its training plan, its movement, its target ranking) and played it: it lost by 173 points per game even though its rules matched 77 % of norxondor’s recorded decisions. The lesson, confirmed tonight, is that a training plan by itself is worthless: what makes a third troll affordable is the *funding* — both trolls collecting one price together, with the orchard supplying the lemons. The same study measured

that funding mechanism at +106 points. Any program built from this report must be judged on real games on the ladder, never on how many decisions it copies.

11. What to try on our own bot, one change at a time

The project's rule is one change per experiment, one hour on the ladder, one reading. In the order of the evidence:

1. **A third troll, funded by both trolls together.** After the second troll, both trolls collect one price — lemons first — and train a carry-3, chop-3, no-harvest lumberjack on the turn it becomes affordable. All four top players do this; our July study measured the funding mechanism at +106.
2. **The orchard rule while funding.** From turn 1, plant lemons and plums on the free cell closest to the shack and the troll, never more than four cells out; bananas after turn 100. This rule fits 78–90 % of more than 20,000 plantings by the four.
3. **An explicit switch.** Produce until the last planned troll is trained, then deforest: every troll chops, no more mining or training (norxondor, Bubaptik).
4. **Plant-and-cut bananas** next to the shack in the deforest phase: one fruit point into four wood points without walking (norxondor: 2,407 own-tree cuts within four turns of planting).
5. **The early raid.** In the first fifty turns, chop the opponent's freshly planted trees near their shack (delineate 56 %, Bubaptik 50 % of their early chop targets). The habit is measured; the trigger is a guess.

12. Sources

Published by the players and the contest's author

- delineate's description of his program (gist, 25 May 2026): <https://gist.github.com/delineate/93ba9d48102e442e764db39d85ac44a3>; his forum posts are in the thread below.
- The contest's "Feedback & Strategies" thread, 35 posts, with the write-ups of wala, laconic_pixel, xSkyline, aangairbender, FinkPloyd, Konstant, putibuzu, Escdemon, Ztrk, eulerscheZahl and Astrobytes: <https://forum.codingame.com/t/spring-challenge-2026-troll-farm-feedback-strategies/208241>
- The contest author's statistics page for all 69 top-league players (trolls trained, trees planted, score over time): https://eulerschezahl.github.io/TrollFarm/troll_stats.html
- The game's rules and referee source code: <https://github.com/eulerscheZahl/Troll-Farm>
- Astrobytes' write-up after the contest (the game's co-author, playing as trlr1990): <https://github.com/Astrobytes/CodinGameTrollFarm/blob/main/README.md>
- MSz's write-ups of his earlier contests (his usual toolkit; nothing about this game): <https://github.com/marekesz/contests>
- Yann Moisan's (yamo's) write-up of 26 May 2026, the design our own bot reproduces: <https://yannmoisan.com>; archived in the repository as docs/reference/yann-mois-an-postmorte-m-2026-05-26.txt.
- Two side threads with rule clarifications: <https://forum.codingame.com/t/208196> and <https://forum.codingame.com/t/208252>.

Our own records and tools (paths in the repository)

- The archived sources, verbatim, and their digest: local_claude_1/reconstructions/sources/ (26 files; SUMMARY.md).
- The recorded games: data/raw/games/ (raw replays with the referee's log; about 24,000 games) and data/processed/ (games, maps and 13.3 million turn records).
- The measured habits: local_claude_1/reconstructions/profiles/ (profile_bot.py, one profile per player, COMPARISON.md).
- The exact replays and the rule tests: local_claude_1/reconstructions/fits/ (reconstruct.py, decision_tables.py, fit_rules.py, one report per player).

- What the project already knew, with its warnings: `local_claude_1/reconstructions/prior-art.md`.
- The four full documents: `local_claude_1/reconstructions/<player>/ALGORITHM.md` for `delineate`, `norxondor_gorgonax`, `Bubaptik` and `MSz`, and the one-page overview `README.md` in the same folder.

13. Small glossary

Word	Meaning in this report
troll	a worker; each player starts with one and can train more
shack	a player's home base, where fruit and wood are dropped to score
talents	a troll's four abilities: speed, carry, harvest, chop, written <code>speed/carry/harvest/chop</code>
train	buy a new troll at the shack, paying fruit and iron by its talents
orchard	trees a player plants next to its own shack, to harvest fruit for prices and to chop later
lumberjack	a trained troll with high carry and chop and little or no harvest power, used mostly for wood
median	the middle value over all games: half the games lie below it, half above
search	a program trying out possible futures before it chooses; <code>delineate</code> does none, <code>MSz</code> perhaps does (a guess)
build-up program	a program that grows to three or four trolls before it goes for wood
raid	chopping the opponent's freshly planted trees near their shack early in the game
plant-and-cut	planting a banana next to the shack and chopping it the next turn: 1 point becomes 4
rating, rank	the ladder's score for a program and its place in the list
replay	the recording of a game, with every command and the referee's log
neural network	a program whose behaviour is a large table of numbers learned from experience, not written rules
rule fit	testing a candidate rule against recorded decisions; the share of decisions it predicts